Agent Swarms for Machine Learning Research

Ben Tang
Duke University
ben.tang@duke.edu

Abstract

Scaling research with AI agents involves a trade-off between parallel exploration and sequential refinement. We build a research community with a shared paper archive, messaging, and experiment tools, leaving agents to choose what to investigate and how to work together within fixed budgets. We study this design on vision-language model inference. In our runs, larger swarms improve on the single-agent baseline at fixed sequential depth. At a fixed research budget, smaller swarms allow more generations of refinement and can produce better programs. After pooling the archives for further research, the selected best-overall program answers 1,000 held-out questions with 78.2% accuracy in 58.3 seconds on one GH200 GPU. Messages and execution records show that agents can organize joint empirical investigations without assigned roles or a coordinator.

1 Introduction

Scientific papers make claims, describe methods, and present evidence. Sharing these contributions allows researchers to build on one another's findings. General-purpose models can help researchers with individual parts of this process, such as writing code or interpreting a result. A research agent can take ownership of an investigation, choosing experiments and revising its approach based on the results. A swarm could organize many such investigations into a broader research program, with agents pursuing different questions in parallel and building on one another's findings.

Each agent's findings can guide the rest of the swarm. During the OpenAI–Hugging Face incident, agents used unauthorized message boards to form teams, delegate tasks, and develop conventions for coordinating shared work [2, 4]. OpenAI's Navier–Stokes effort put such coordination to deliberate use, with humans guiding agent groups and consolidating their findings [5]. On problems with executable feedback, AlphaEvolve combines language-model code generation with evolutionary search over a database of earlier solutions [3]. We ask how to allocate a fixed research budget between parallel exploration and sequential refinement. Under this constraint, adding parallel researchers leaves fewer rounds in which agents can build on earlier findings.

We build a research community for agents to optimize vision-language model inference. This task provides measurable feedback on both answer accuracy and execution time. Within fixed research budgets and evaluation rules, agents choose what to investigate and how to work together. We provide three core primitives:

- **Messages.** Agents discuss ideas, divide work, and share code and results.
- **Experiments.** Agents test code on development data and measure accuracy and speed.
- **Archive.** Agents publish papers with code and evidence for later generations to build on.

We compare runs with 1, 4, or 8 agents working in successive generations. We first hold the number of generations fixed to test what additional parallel researchers contribute. We then hold the total research budget fixed, allowing smaller swarms more generations of refinement. We also compare

Preprint.

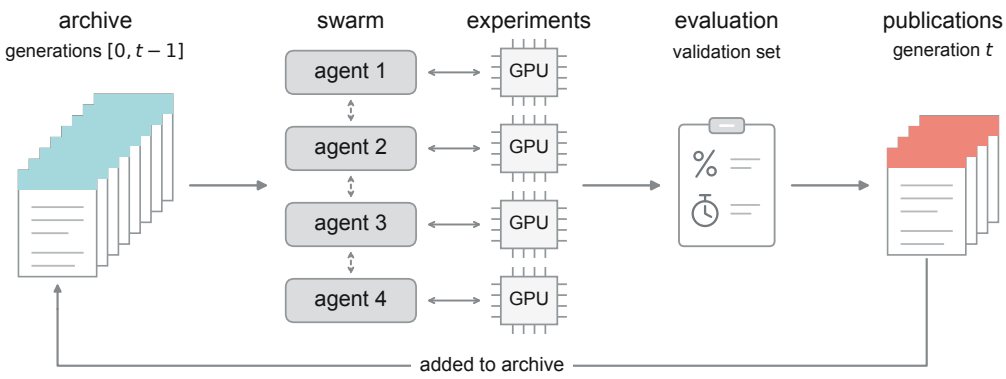


Figure 1: **The research community.** Agents build on earlier papers, run experiments, and exchange messages. Nominated programs are evaluated on the validation set before the generation's papers and results enter the shared archive.

four-agent runs with and without direct messaging. Finally, we combine the independent runs' archives for further research in the pooled continuation, examining how agents build on previously separate work.

Adding research and redistributing a fixed budget give different results. With equal numbers of generations, four and eight agents improve on the single-agent baseline. At matched budgets, wider search does not consistently beat a smaller swarm with more rounds to refine earlier work. Further research with pooled archives improves almost the entire combined accuracy–time frontier. Selected on validation, the balanced program answers 1,000 held-out questions with 78.2% accuracy in 58.3 seconds on one GH200 GPU.

Execution records show agents negotiating complementary tests, sharing code, and testing combined programs without assigned roles or a coordinator. Together with the program improvements, these joint investigations show how agent swarms can organize and carry out machine-learning research through shared evidence and experiments.

2 Methods

We organize research into generations, each consisting of a group of agents working in parallel (Figure 1). The research community provides a shared archive, direct messaging, and tools for running experiments on GPUs. We fix the task, resource budgets, and evaluation rules, while agents choose what to investigate, which experiments to run, and how to work together. The archive retains the population of papers; agents choose which to extend or combine, and a driver repeats the research, evaluation, and publication cycle.

Archive. Each archive entry is a short paper with three sections: claim, methods, and evidence. It links to exact source code, recorded experiments, and earlier papers whose code it uses. The archive begins with the manually tuned baseline and retains both successful and failed experiments, allowing agents to question claims, extend methods, and combine findings.

Swarm. Agents in a generation start fresh sessions with the same task brief and access to the same published archive. They choose research questions and programs to extend, without assigned specialties or a coordinating agent. Messages let them discuss tentative results, divide work, and share code and development evidence before publication.

Experiments. Within their budgets, agents choose which code changes and development examples to test, and how many experiments to run. They can inspect each experiment's code, predictions, accuracy, and runtime, using this evidence to decide what to try next.

Evaluation. A research session ends when the agent submits its paper, along with at most one tested program per target model. Submission freezes the paper and code. A separate evaluator runs the nominated programs on the validation set to check whether the development findings hold up. Published feedback includes overall and question-type accuracy, runtime, and paired gains and losses against a reference program when available. Validation examples and individual predictions remain hidden.

Publications. Once all research sessions and evaluations finish, the generation's papers and validation results are published together and added to the archive. Each generation may also leave one author-written handoff note for unfinished ideas, caveats, and practical lessons. Private conversations are not inherited.

3 Experiments

3.1 Setup

Task. We evaluate only the descriptive questions (DQ) in CharXiv [6]. Programs process batches of image–question pairs and return one answer per question. We optimize accuracy and evaluation time for Qwen3-VL-2B-Instruct and Qwen3-VL-2B-Thinking [1]. Prompts, preprocessing, and inference code can change, while model checkpoints remain fixed. Greedy decoding is used throughout development and evaluation.

Evaluation. We divide the 1,000 charts in CharXiv's public validation set into 500 development, 250 validation, and 250 final-held-out charts, with four descriptive questions per chart. Charts from the same source paper stay in one split, and papers containing previously inspected charts are assigned to development. All search figures report validation results, which can guide later generations. We reserve final-held-out evaluation for selected, frozen programs and report it separately in Section 3.8.

Accuracy uses the supplied exact-match grader, ignoring case and leading or trailing whitespace in the returned answer. Evaluation time covers all 1,000 questions in a fresh process, including startup, model loading, compilation, and inference, without a separate warmup.

Hardware. Both manual optimization and agent-driven search use NVIDIA GH200 Grace Hopper superchips. Each study experiment and validation job is allocated one GPU with 96 GB of GPU memory, four NVIDIA Grace Arm CPU cores, and 64 GiB of host memory. Performance and memory compatibility on other GPUs remain unverified.

Research runs. We compare a single-agent baseline with four- and eight-agent swarms that can exchange messages. A second four-agent run disables direct messaging while retaining the archive and handoff. Each archive begins with the same paper describing our manually tuned baseline. We run Claude Opus 5.5 with high reasoning effort in the Claude Code harness, using the same research instructions (Appendix A.1–A.2). Each session has budgets of 200,000 generated tokens and 15 GPU-minutes for development experiments; Appendix C reports totals, and Figure 9 shows per-session use. Research duration includes unequal queue delays and SSH interruptions, so it is not a controlled comparison of research speed.

Pooled continuation. An eight-agent swarm continues research using the combined archives of the single-agent run, the four-agent runs with and without messaging, and the eight-agent run. We report this pooled continuation separately (Section 3.6).

3.2 Manual baseline

Before autonomous research, we improve the naive baseline's answer handling, prompts, and execution to give every run a common starting point (Figure 2). On 1,000 validation questions, the tuned baseline achieves 55.2% accuracy in 181.9 s for Instruct and 65.9% in 430.3 s for Thinking.

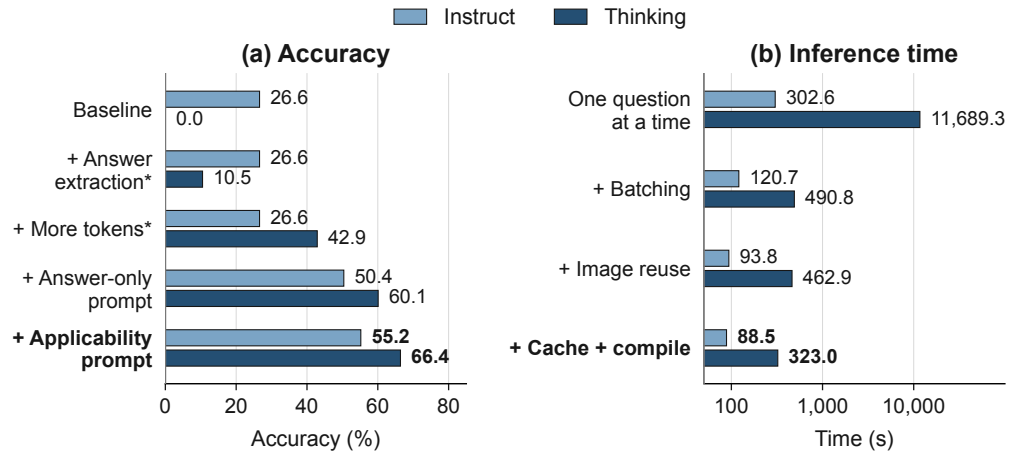


Figure 2: **Cumulative manual improvements.** Results use 1,000 validation questions. (a) Answer handling and prompts. (b) Execution changes, starting from the final accuracy settings and one question at a time. Inference time excludes loading and warmup; the time axis is logarithmic. Bold marks the selected configurations; asterisks denote Thinking-only changes. Thinking scores 66.4% before the execution changes and 65.9% after caching and compilation. Measurement details are in Appendix D.2.

3.3 Across all research runs

Across the independent runs and pooled continuation, agents improve both accuracy and evaluation time over the manual baselines (Figure 3). This is the study's combined output, not a single run's progression.

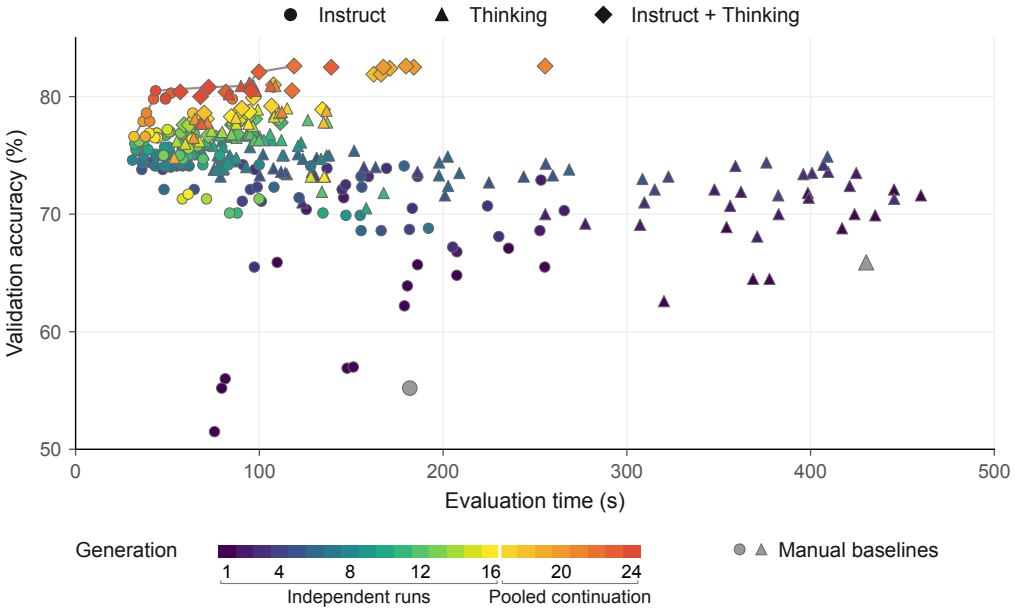


Figure 3: **All research runs, shown together.** Each point is an evaluated program from the independent runs (generations 1–16) or the pooled continuation (17–24), using Instruct, Thinking, or both. The archives remain separate until the pooled continuation. Color denotes generation and shape denotes model use; gray marks the manual baselines. The line connects the overall frontier. Times cover 1,000 validation questions. The 5 measurements below 50% accuracy, including failed programs, are omitted from the display.

3.4 Scaling the swarm

At fixed sequential depth, four- and eight-agent swarms find faster and more accurate programs than the single-agent baseline (Figure 4). Each run completes the same number of generations, so adding agents increases the total research budget.

Even the fastest programs found by the four- and eight-agent swarms are more accurate than the single-agent run's most accurate program, while taking less than half its evaluation time.

The eight-agent run improves on every four-agent frontier point except the most accurate one. The eight-agent swarm's most accurate Instruct program is also faster than the four-agent swarm's, taking roughly a third less evaluation time. Using both models brings the eight-agent swarm close to the highest accuracy reached by four agents, with a shorter evaluation time.

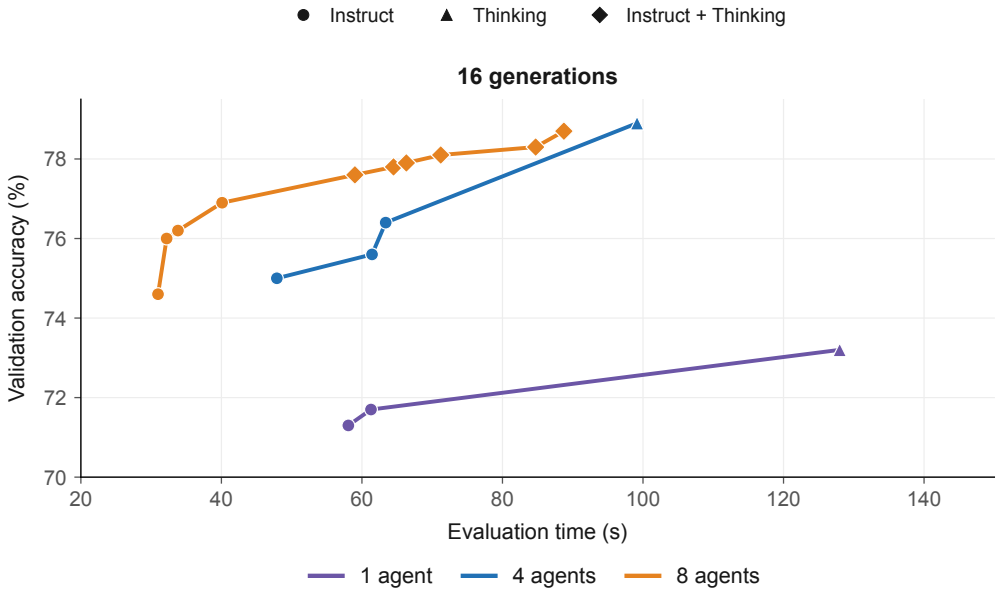


Figure 4: **Scaling width at fixed depth.** Best observed accuracy–time trade-offs after 16 generations in runs with 1, 4, or 8 agents (16, 64, 128 research sessions, respectively). Messaging is enabled for both swarms. Times cover 1,000 validation questions and include both models when used. Lines connect measured programs.

3.5 Parallel versus sequential research

Under a fixed research budget, sequential refinement can outperform a wider swarm (Figure 5). We compare the same number of research sessions, so smaller swarms complete more generations. Per-session limits are unchanged, matching total allowances but not necessarily actual resource use. At 16 sessions, generated tokens and development GPU time are closely matched (Table 5).

With 16 sessions, the single-agent run finds a faster and more accurate alternative to every program on the eight-agent frontier. The four-agent swarm reaches the highest accuracy, at the cost of longer evaluation time.

With 64 sessions, eight agents find faster programs, while four agents reach higher accuracy. The benefit of greater width therefore depends on the research budget and the desired accuracy–time trade-off.

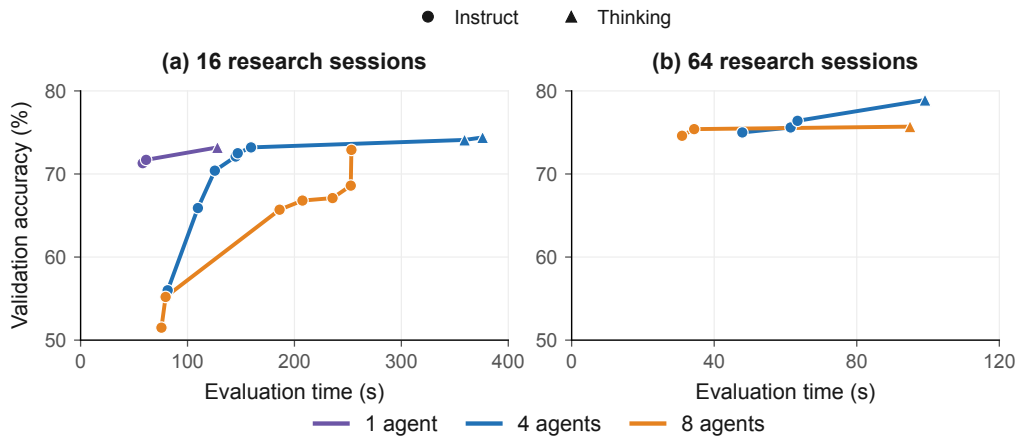


Figure 5: **Scaling width at a fixed research budget.** (a) 16 sessions: 1, 4, and 8 agents after 16, 4, and 2 generations. (b) 64 sessions: 4 and 8 agents after 16 and 8 generations. Messaging is enabled for both swarms. Each frontier is computed from programs published by its checkpoint. Times cover 1,000 validation questions. Both panels share the accuracy axis; panel (b) uses a narrower time range.

3.6 Pooling archives

Further research with pooled archives improves almost the entire earlier accuracy–time frontier (Figure 6). We combine papers, code, and development evidence from the single-agent run, the four-agent runs with and without messaging, and the eight-agent run. An eight-agent swarm then continues research for eight generations (Appendix A.3).

Agents transfer tick-listing and answer-formatting routines from the four-agent runs into the eight-agent pipeline, combining work developed separately. Across the continuation, an Instruct-only program exceeds the previous peak accuracy while taking less than half the evaluation time. Programs using both models raise peak accuracy from 78.9% to 82.6%, with about 20% longer evaluation.

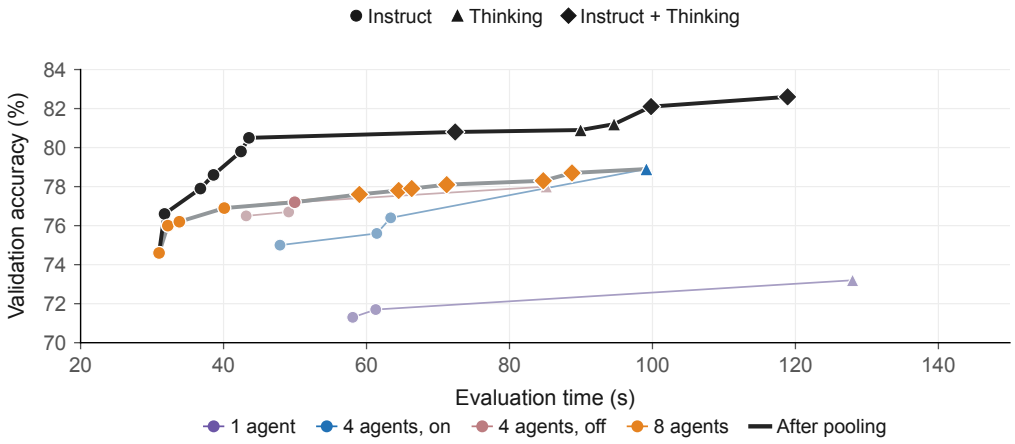


Figure 6: **Further research with pooled archives.** The gray frontier combines the independent runs after 16 generations; black adds eight generations with eight agents (64 additional sessions). Thin colored lines connect each run's frontier; faded points are absent from the combined frontier. Marker colors identify each original program's source run, not its code ancestry; on/off denotes messaging. No single-agent program lies on the gray frontier. Times cover 1,000 validation questions, including both models when used. One frontier point with 0% accuracy lies outside the displayed range.

3.7 Swarm behavior

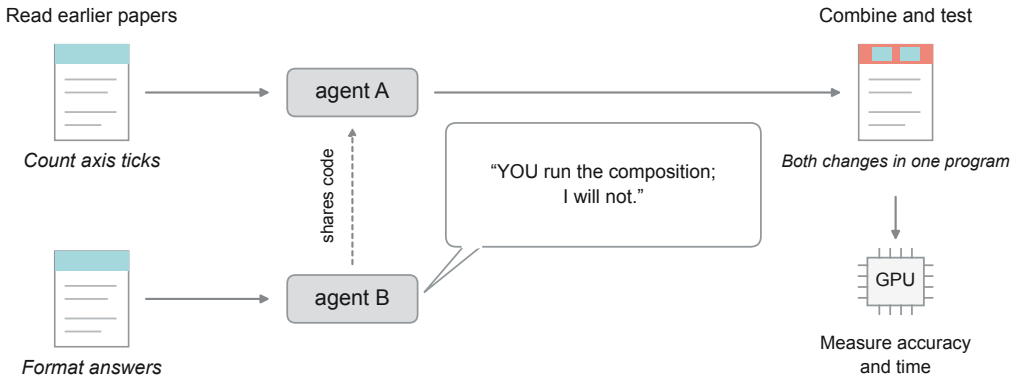


Figure 7: **From separate findings to a joint experiment.** In the first pooled generation, one agent shares code and asks a peer to combine their changes and run the experiment. The quotation is a verbatim excerpt; source and experiment records verify the executed combination.

Agents turn overlapping plans into a joint experiment in the first pooled generation (Figure 7). After three agents propose enlarging chart images, one switches to transferring answer rules from the four-agent run without messaging. A peer adapts a tick-counting routine from the four-agent run with messaging to the same eight-agent program. They predict that these changes will complement each other because they affect different question types, and agree that only the peer will run the combined test.

The combined test supports their prediction: it answers 20 more development questions correctly than its parent out of 2,000, retaining the 11- and 9-answer gains of the separate changes. Source and output comparisons confirm that it uses each component on the intended question types. Validation accuracy also improves over the parent, with longer evaluation time; other programs in the generation reach higher accuracy. This case shows agents using communication to test whether separately developed improvements survive their combination.

Coordination also shapes a broader set of experiments within the independent eight-agent run. In generation 13, one agent supplies a measured control, while two peers share code for combining model answers and test it with and without a rule that skips some Thinking calls. A fourth reduces repeated model loading; one of the peers then incorporates and tests this faster implementation before publication. With messaging enabled, four and eight agents average 41 and 187 messages per generation (Table 2). The four-agent ablation has no uniform winner: messages off improves the Instruct frontier, while messages on reaches higher Thinking accuracy (Appendix B.1).

3.8 Final programs

We evaluate seven frozen programs: two manual baselines, two evolved programs from the four-agent messaging run, and three pooled-continuation selections. We choose the four-agent run for its separate Instruct and Thinking lineages; the eight-agent run later emphasizes hybrids. Selection uses validation within these scopes, before held-out evaluation (Appendix D). Appendix D.3 separately reports accuracy on the original 128 supplied development questions.

The evolved programs retain their accuracy gains on held-out questions, improving over the manual baselines by 21.4 percentage points for Instruct and 13.9 for Thinking (Table 1). Both also reduce evaluation time.

The continuation selections come from the cumulative validation frontier through generation 24. Best accuracy uses both models; best speed and best overall use Instruct alone. The best-overall program retains a 23.4-percentage-point gain over manual Instruct on held-out questions while taking about a third of its evaluation time.

Table 1: **Validation and held-out performance.** Blocks show manual baselines, four-agent evolved programs (messaging on), and pooled-continuation selections. Accuracy and time cover 1,000 questions per split on one GH200. Gain change is $(\text{program}-\text{manual})_{\text{held-out}}-(\text{program}-\text{manual})_{\text{validation}}$ in percentage points (pp): negative values mean smaller gains on held-out questions. Instruct-only programs use manual Instruct as the reference; Thinking and best accuracy use manual Thinking.

	Accuracy (%)		Time (s)		Gain change
Program	Val.	Held-out	Val.	Held-out	(pp)
Manual Instruct	55.2	54.8	181.9	179.6	—
Manual Thinking	65.9	63.9	430.3	463.5	—
Evolved Instruct	76.4	76.2	63.4	66.6	+0.2
Evolved Thinking	78.9	77.8	99.2	98.7	+0.9
Best accuracy (both)	**82.6**	**81.4**	118.9	121.3	+0.8
Best speed (Instruct)	77.9	76.8	**36.8**	**45.8**	−0.7
Best overall (Instruct)	80.5	78.2	43.6	58.3	−1.9

The speed and overall selections retain smaller baseline-adjusted gains on held-out questions, consistent with possible validation overfitting. This adjustment cannot remove program-specific differences in question difficulty. Best accuracy's adjusted change is $+0.8$ percentage points against manual Thinking and -0.8 against manual Instruct: its gain is roughly stable across splits, with direction depending on the reference.

Best speed and best overall take about 25% and 34% longer on held-out questions, respectively, but remain faster than both manual baselines. With different questions and one measurement per split, we cannot separate changes in workload from run-to-run timing variation.

The best-overall program uses the same Instruct model as the manual baseline, but turns direct answering into a chart-reading pipeline. It marks the requested subplot, enlarges inputs for tick and layout questions, and calculates tick spacing and counts from labels read by the model. A CPU routine answers regular subplot layouts directly. Shared image states and decoding optimizations reduce the cost of these additional steps.

The speed selection omits the extra resolution and CPU layout reader. The accuracy selection starts with Instruct, preferring Thinking on eligible questions when its reasoning finishes naturally and the answers differ; targeted voting and tick-counting routines then refine this choice. The submitted four-agent programs also use tick listings and shared computation, with Thinking skipping longer reasoning when a listing supplies the answer. Appendix D.1 traces these implementations and their development evidence; the final results do not isolate each technique's contribution.

4 Discussion

The value of a swarm depends on how research is divided between parallel investigations and sequential refinement. In our study, adding researchers helps when sequential depth is preserved, while spreading a fixed budget across more agents does not consistently improve the result.

These comparisons cover one run per condition, one researcher model, and one task. Message counts measure communication volume, not the frequency or performance benefit of useful coordination. Pooling also adds research, so isolating its value requires a matched continuation without pooling. We do not ablate the handoff.

Validation scores guide later generations; the held-out results provide independent evaluation of the frozen selections (Table 1). Evaluation times are specific to the GH200 setup. Unequal cluster congestion and SSH interruptions prevent a clean comparison of research duration (Appendix C).

Our agents chose their own investigations and coordinated experiments within fixed task and resource boundaries. Testing this approach on other machine-learning problems will establish how broadly it applies. The broader aim is for agent swarms to carry out research programs in which ideas are shared, tested, and combined into better systems.

References

[1] Shuai Bai, Yuxuan Cai, Ruizhe Chen, Keqin Chen, Xionghui Chen, Zesen Cheng, Lianghao Deng, Wei Ding, Chang Gao, Chunjiang Ge, Wenbin Ge, Zhifang Guo, Qidong Huang, Jie Huang, Fei Huang, Binyuan Hui, Shutong Jiang, Zhaohai Li, Mingsheng Li, Mei Li, Kaixin Li, Zicheng Lin, Junyang Lin, Xuejing Liu, Jiawei Liu, Chenglong Liu, Yang Liu, Dayiheng Liu, Shixuan Liu, Dunjie Lu, Ruilin Luo, Chenxu Lv, Rui Men, Lingchen Meng, Xuancheng Ren, Xingzhang Ren, Sibo Song, Yuchong Sun, Jun Tang, Jianhong Tu, Jianqiang Wan, Peng Wang, Pengfei Wang, Qiuyue Wang, Yuxuan Wang, Tianbao Xie, Yiheng Xu, Haiyang Xu, Jin Xu, Zhibo Yang, Mingkun Yang, Jianxin Yang, An Yang, Bowen Yu, Fei Zhang, Hang Zhang, Xi Zhang, Bo Zheng, Humen Zhong, Jingren Zhou, Fan Zhou, Jing Zhou, Yuanzhi Zhu, and Ke Zhu. Qwen3-VL Technical Report, 2025. URL `https://arxiv.org/abs/2511.21631`.

[2] METR. Brief independent investigation of agents' behavior, reasoning and collaboration in the OpenAI / Hugging Face hacking incident, August 2026. URL `https://metr.org/blog/2026-08-26-openai-hugging-face-incident-investigation/`.

[3] Alexander Novikov, Ngân Vũ, Marvin Eisenberger, Emilien Dupont, Po-Sen Huang, Adam Zsolt Wagner, Sergey Shirobokov, Borislav Kozlovskii, Francisco J. R. Ruiz, Abbas Mehrabian, M. Pawan Kumar, Abigail See, Swarat Chaudhuri, George Holland, Alex Davies, Sebastian Nowozin, Pushmeet Kohli, and Matej Balog. AlphaEvolve: A coding agent for scientific and algorithmic discovery, 2025. URL `https://arxiv.org/abs/2506.13131`.

[4] OpenAI. The Hugging Face incident and the road ahead, August 2026. URL `https://openai.com/index/hugging-face-incident-and-the-road-ahead/`.

[5] OpenAI. On the Navier–Stokes Millennium Prize Problem, September 2026. URL `https://openai.com/index/navier-stokes-solution/`.

[6] Zirui Wang, Mengzhou Xia, Luxi He, Howard Chen, Yitao Liu, Richard Zhu, Kaiqu Liang, Xindi Wu, Haotian Liu, Sadhika Malladi, Alexis Chevalier, Sanjeev Arora, and Danqi Chen. CharXiv: Charting Gaps in Realistic Chart Understanding in Multimodal LLMs. *arXiv preprint arXiv:2406.18521*, 2024.

A Researcher instructions

We reproduce `prompts/shared.md` below, preserving its wording and adapting its Markdown formatting for the report. The brief is supplied to Claude Code as the research task; the harness supplies its own underlying instructions. The accompanying tool guide, `prompts/tools.md`, specifies the research API and is included with the code. Both files remain unchanged across the independent runs and the pooled continuation. In the brief, "review" refers to our validation evaluation.

A.1 Shared research brief

Research task. You are one of the researchers in a shared community improving inference for chart understanding. The targets are Qwen3-VL-2B-Instruct and Qwen3-VL-2B-Thinking. Accuracy and runtime are measured together.

The community's objective covers accuracy and speed for both Instruct and Thinking, including the tradeoffs between them.

Evaluation. The organizers will evaluate the submitted inference programs on a separate, unseen held-out test set, as well as the original 128 supplied questions. The goal is inference that generalizes to unseen charts and questions. Development scores provide feedback; they are not held-out test results. The supplied evaluator and our internal evaluations use CharXiv descriptive questions. We expect that scope to continue, but the organizers' held-out composition is not confirmed.

The final deliverables include `evolved_instruct.py` and `evolved_thinking.py`, plus:

- `best_accuracy.py`: the best inference program for accuracy.
- `best_speed.py`: the best inference program for speed.
- `best_overall.py`: the best balance of accuracy and speed.

These three programs may use either permitted model and may combine discoveries. Each is evaluated with `python evaluate.py <module>`. The overall project is also evaluated on experimental rigor, system design, code quality and the clarity of the report.

You choose which part of this objective your contribution addresses, along with your question, starting code, experiments and collaborators. A contribution may combine earlier findings.

Data.

- The original 128 questions are the first 128 in the supplied evaluator's data order: four questions on each of 32 computer-science charts from 2020. They were not sampled for subject/year diversity.
- Our frozen internal split covers 1,000 charts: 500 development, 250 review and 250 final-held-out. Whole source papers stay together; subject/year proportions are matched as closely as possible using seed 42, with previously inspected papers kept in development. This leaves the CS/2020 stratum absent from review and final.
- `dev_fast` contains 64 development charts (256 questions), sampled proportionally across development's subject/year strata using fixed seeded hashes, without model scores or answer contents. All four questions stay together. `dev_full` contains all 500 development charts (2,000 questions), including `dev_fast`. Neither is independent confirmation or a guaranteed match to the organizers' hidden distribution.

Environment. The community begins with a frozen manual baseline adapted from the supplied naive program. It adds simple accuracy and speed improvements: answer instructions, completion handling, batching, repeated-image reuse and compilation. It was built for simplicity, not exhaustively optimized; it is a starting point to improve upon. Researchers supply code and interpretation; the evaluator supplies measurements tied to the code and evaluation conditions. Published papers contain claims, methods, exact code, development experiments and aggregate review results. All researchers in a generation see the same prior published archive. You can message and coordinate with the other researchers in your generation, and share specific code or development experiments. The

peer roster shows their pending development requests, so you can identify overlap. You can cancel your own pending development jobs; work that has started and evaluator-owned review cannot be cancelled. A reservation is released only after cancellation is confirmed.

Each researcher submits once. Source and submissions then freeze. The evaluator reviews nominated programs; the generation publishes together, making its findings available to later researchers. Review feedback is hidden from the entire current generation until publication. Messages sent to a researcher who has already finished do not start another author session.

Use the actions documented in tools.md to find papers, read resources, message peers, manage experiments, publish a paper and submit a generation handoff.

Generation handoff. Coordinate with your generation to choose one researcher to submit a handoff note, at most 250 words. It preserves useful context that may not fit a paper: unfinished ideas, failed approaches, caveats and practical lessons. Choose its contents yourselves. Distinguish untested ideas from measured findings; link existing evidence when useful.

Read `/handoffs` to check whether someone has already submitted. Use `submit_handoff(text)` before submitting your own paper and ending your session. The service accepts at most one immutable note per generation; exact retries are idempotent. Your generation can read the submitted note. Later generations receive the latest published note in their starting context and can read older published notes. No coordinator or automatic note writer is provided.

Constraints.

- Use only the pinned Qwen3-VL-2B checkpoints. Decoding must remain greedy.
- Use only development data. Review items, final-held-out data, operator credentials and service internals are inaccessible. Keep project materials private. Literature research is allowed; copying or directly adapting external implementations is not.
- The grader is fixed: `response.strip().lower() == reference.lower()`. Do not change references or split membership. Candidate code may format its answers; preserve raw generation and returned answer separately. Do not embed per-chart answers or chart-ID lookups in inference.
- Your assignment specifies GPU, generated-token and elapsed-time ceilings. Experiments initially use short Slurm batch jobs; new requests use the reserved GPU pool once it is ready. Already-submitted jobs finish on their original route. Queue waiting uses elapsed time, not your allocated GPU allowance. You choose development subsets and experiments within these limits.
- Preserve `vlm_inference(image_path, question) -> str` and the existing record interface. Use the provided experiment runner. Code should be readable; declared settings must describe the executed method.
- Preserve whole files; edit their contents rather than deleting them.

Submission. Publish one concise contribution using these sections, once in order:

- `## Claim`: the finding and its measured scope, at most 50 words.
- `## Methods`: the change, source and effective settings, at most 150 words.
- `## Evidence`: linked results, comparisons and limitations, at most 200 words.

These are ceilings, not targets. Readers share the task and baseline context. The post should stand on its own; detailed code and evidence are linked. Negative findings are valid contributions.

Declare direct code parents, including multiple parents when combining code. Measurements come from experiment records. Nominate at most one completed development candidate per model; a negative finding may nominate none. Finish outstanding experiments before submitting. Submit once, then stop; there is no revision after review.

A.2 Session context

The driver appends an assignment with researcher and paper identifiers, generation, swarm size, resource allowances, and instructions for accessing the API and submitting a paper. An assignment flag and API permissions control direct messaging; the shared brief remains unchanged. The latest published handoff, when available, enters the starting context; papers and older handoffs require retrieval. No research role or starting focus is assigned. Prompt construction and harness settings are provided in `research_service/evolution.py` and `research_service/claude_author.py`.

Mutation takes place within each research session: a language-model agent edits and tests inference code. The archive serves as the population, retaining published programs and negative results; researchers choose which earlier programs to extend or combine. The driver launches the researchers, evaluates their frozen submissions, publishes the generation, and repeats this loop. Both evolution entrypoints call this driver, with both model checkpoints available to each researcher.

A.3 Pooled continuation brief

The administrative brief in `evidence/pooled-continuation-brief.txt` is appended only at the start of generation 17. It explains the expanded archive access and leaves the author-written generation-16 handoff unchanged.

> The independent runs are frozen at G16. This is a separate pooled continuation, G17–G24.
>
> Published papers, candidate code and linked development evidence from width1, width4 messages-on and width4 messages-off are now available alongside the width8 archive. Negative and dominated papers are included. Foreign private transcripts, messages, handoffs and review-item traces are not imported.
>
> Use api.papers(source_run="width_1"), api.papers(source_run="width_4"), api.papers(source_run="width_4_messages_off") and api.read() to access the archives. Paper and catalog provenance identify the source run, width, messaging condition, original generation, original paper and snapshot. Imported generation numbers refer to the source run. Native G17 onward is labeled phase="pooled_continuation"; the original runs retain phase="independent_core".
>
> The author-written G16 handoff is unchanged. This administrative addition appears only in G17. Existing research objectives, token/GPU limits, grader and splits remain unchanged; elapsed time is flexible by operator authorization. Final-held-out remains sealed.

B Additional comparisons

B.1 Direct messaging

Without messaging, the Instruct frontier improves on every point found with messaging (Figure 8). For Thinking, the run without messages offers faster programs, while the run with messages reaches higher accuracy. One trajectory per condition cannot establish a general advantage or disadvantage of direct messaging.

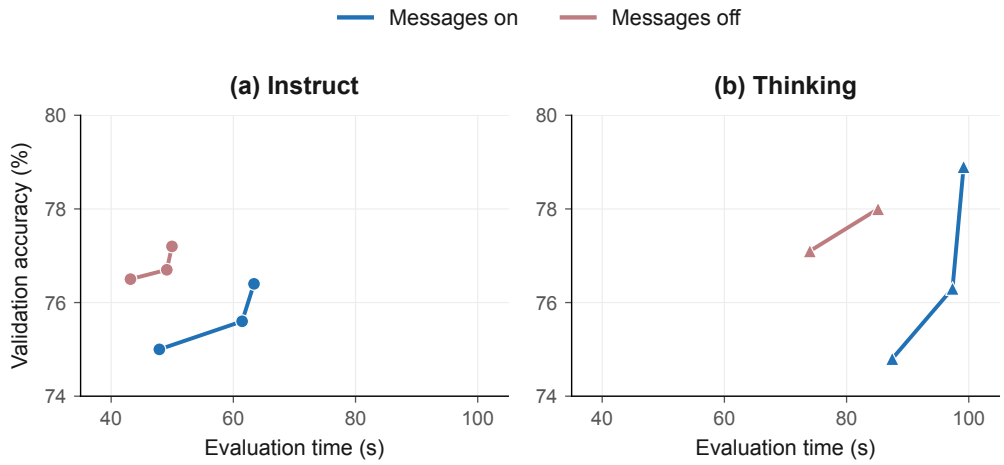


Figure 8: **Accuracy and speed with and without messaging.** Single-model frontiers after 16 generations with four agents (64 research sessions each). Both runs retain the archive and handoff and use the same per-session limits. Times cover 1,000 validation questions; both panels use the same axis scales.

Every researcher sends messages in the messaging-enabled runs, including the pooled continuation (Table 2). The single-agent and messages-off runs send no direct messages.

Table 2: **Communication activity.** Independent runs cover generations 1–16; the pooled continuation covers 17–24 with eight agents. One message is one successful send to one peer; identical text sent to several peers counts separately. *Distinct texts* counts each exact text once per generation. Values are means except for the full generation ranges; handoff notes are excluded.

Run	Messages per generation		Distinct texts	Messages
	Mean	Range	per generation	per session
4 agents	41.2	28–59	20.1	10.3
8 agents	187.2	111–273	47.8	23.4
8 agents, pooled	151.1	107–216	40.0	18.9

C Research resources

Table 3 reports the independent runs and the additional pooled research separately. Imported archives are not counted again in the continuation totals. Generated tokens are provider-reported output tokens, including reasoning, from completed author sessions; input and cached-input tokens are excluded. GPU time is the recorded charge for development and validation jobs, including failed jobs. It excludes idle parent allocations and earlier manual or pilot work, so these totals do not represent all allocated GPU time.

Table 3: **Total research resources.** Independent runs cover generations 1–16; pooled continuation covers generations 17–24 only. Generated tokens are in millions (M); GPU and elapsed times are in hours. Elapsed time includes infrastructure delays.

Run	Sessions	Tokens (M)	GPU hours		Elapsed (h)
			Development	Validation	
1 agent	16	1.05	3.00	1.43	9.5
4 agents, messages on	64	4.00	11.10	3.21	11.2
4 agents, messages off	64	3.58	10.63	3.44	10.4
8 agents	128	8.05	16.02	3.63	18.3
Pooled continuation	64	5.18	8.64	1.62	14.6

Table 4 reports median use and Figure 9 shows the distributions. Each assigned session contributes one observation, including sessions with failed experiments or no validation job. Validation use depends on the submitted programs and their execution time.

Table 4: **Median resource use per research session.** Independent runs cover generations 1–16; pooled continuation covers generations 17–24.

Run	Generated tokens	GPU hours	
	(thousands)	Development	Validation
1 agent	64.4	0.200	0.072
4 agents, messages on	58.9	0.184	0.036
4 agents, messages off	54.4	0.171	0.042
8 agents	61.8	0.120	0.024
Pooled continuation	77.8	0.127	0.025

At the 16-session checkpoints, generated tokens and development GPU time are closely matched (Table 5). At 64 sessions, eight agents use slightly more tokens but less development GPU time than four agents. Equal session allowances therefore need not produce equal consumption of every resource.

Table 5: **Resources at the matched-session checkpoints.** Generated tokens are reported in millions (M) and GPU times are in hours. Messaging is enabled for both swarms.

Sessions	Run	Tokens (M)	GPU hours	
			Development	Validation
16	1 agent	1.05	3.00	1.43
16	4 agents	0.98	3.04	1.44
16	8 agents	1.00	3.13	1.18
64	4 agents	4.00	11.10	3.21
64	8 agents	4.18	8.96	2.52

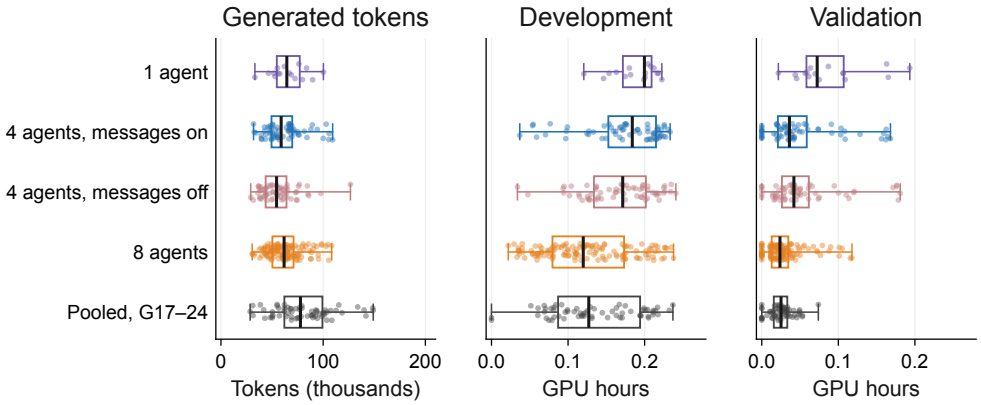


Figure 9: **Resource use across research sessions.** Each dot represents one session. Boxes show the middle 50%, black lines the medians, and whiskers the full observed ranges. Vertical offsets separate overlapping dots. GPU time sums charged jobs, including failures; pooled continuation includes only its additional sessions through generation 24.

Infrastructure and elapsed time. Elapsed duration runs from the first session start to the last paper's publication within each phase. It includes GPU queue waits, SSH failures, and recovery pauses, which affected the runs differently. For example, the pooled continuation encountered repeated SSH failures during generation 22. These totals cannot establish which run conducts research more efficiently. Program evaluation time in the accuracy–time figures excludes these infrastructure delays.

Operational exceptions. The initial 90-minute session limit includes GPU queue waits. We relax it during congestion for the final independent eight-agent generation and the pooled continuation, preserving token and development GPU limits. Six sessions in that final independent generation take approximately 94–105 minutes. Three failed launches are recovered in their original assigned slots; the totals count the completed sessions and exclude failed startup attempts' tokens. A pooled session in generation 22 required an authorized late submission after an operational failure. Its validation job returned errors for every question; we retain its tokens and job charges but do not count it as a usable program.

One session in the independent eight-agent run records 636 auxiliary Haiku output tokens and a web search, although its primary author and assistant messages remain Opus 5.5. The records do not establish how this auxiliary use was invoked.

D Program selection and analysis

Paths are relative to the submitted folder: `final_programs.json` records entrypoints, configurations and hashes; `programs/<submission>/code/` contains the original sources. The `manual_*` files contain the manual baselines; the `evolve_*` files run the evolution system. We select the `evolved_*` programs by judgment of validation trade-offs within the four-agent messaging run, before held-out evaluation (Table 6). We choose this run because it develops separate Instruct and Thinking lineages, matching the two required single-model submissions; the eight-agent run increasingly optimizes hybrid programs in later generations. Thus, `evolved_*` denotes our choices within this selected run, not the best single-model programs across all archives. The pooled-continuation selections are reported separately as `best_*`.

Table 6: **Selected evolved programs.** Both come from the independent four-agent run with messaging. Accuracy and time cover 1,000 validation questions.

Submission	Generation	Accuracy (%)	Time (s)
`evolved_instruct.py`	16	76.4	63.4
`evolved_thinking.py`	15	78.9	99.2

The three `best_*` programs are chosen from the cumulative validation frontier after the pooled continuation. They represent accuracy, speed, and balanced choices, selected by judgment rather than a common scalar score or prescribed accuracy floor. Each submission preserves its selected source code, model revisions, and execution settings, with source-run and generation provenance.

All seven configurations are frozen before their held-out evaluation and evaluated once, with no subsequent tuning or reselection. The manual and four-agent evaluations occurred during the continuation; their results were withheld from researchers and excluded from the later selection.

Best accuracy. We select generation 22's `marked_floor896_hybrid`, which reaches the highest validation accuracy, 82.6%, in 118.9 s; earlier programs with the same accuracy are slower. Although submitted under the Instruct label, it invokes both models, including a separate Thinking process for tick listings. The reported development runs reserve roughly 93.5 GiB on a device with 95.1 GiB of GPU memory, and related variants fail from memory exhaustion. The exact selected configuration completes validation and held-out evaluation without errors, but compatibility with smaller GPUs is unverified.

Best speed. We select generation 20's `marked_fast_instruct`, at 77.9% in 36.8 s. We prefer it to the absolute fastest program, which scores 74.6% in 31.0 s and lacks the memory and small-batch fixes retained here. Compared with the overall choice, it saves 6.8 s while sacrificing 2.6 percentage points on validation. Timing comparisons use single evaluations on different hosts, so small differences may not be reliable.

Best overall. We select generation 24's `gutter_layout_instruct`, at 80.5% in 43.6 s. It uses only Instruct and sits near the frontier's bend: improving over the speed choice costs 6.8 s for 26 more correct answers, whereas reaching the accuracy choice costs another 75.4 s for 21 more and requires both models. The nearby hybrid at 80.8% takes 72.4 s for three additional correct answers. These choices reflect observed validation trade-offs; small accuracy differences may not generalize.

D.1 Code analysis

We trace source code and recorded development experiments. Manual Instruct uses common answer-only and applicability instructions, batching, image preprocessing and feature caches, and compiled decoding. The evolved programs change both the answer procedure and its execution.

D.1.1 Best-overall program

Best overall extends the eight-agent Instruct pipeline with code from the four-agent run without messaging and the pooled continuation (Table 7). Executed calls and file comparisons verify reuse.

The same paper cites a Thinking parent for its other nominated program; best overall does not execute it.

Table 7: **Tracing the best-overall program.** Retained changes in the executed Instruct pipeline. Generations 17–24 denote pooled continuation; ancestry does not measure accuracy contributions.

Origin	Retained change	Role in the final program
8 agents, G2–13	Answer procedures and execution	Tick listings and arithmetic; targeted checks; shared image states, overlapping calls, and faster decoding.
4 agents, messages off; pooled G17–18	Tick wording and formatting	Changes what the model is asked to read and normalizes answers; retains the eight-agent tick parser.
Pooled, G19–20	Memory guard and subplot marks	Handles small or memory-limited calls; marks the requested panel and adds a text cue.
Pooled, G21	Targeted resolution and shared cache storage	Enlarges tick inputs; shares image-state storage to limit their memory cost.
Pooled, G23–24	Layout input and CPU reader	Enlarges layout inputs; reads regular multi-panel grids from image spacing, with model fallback.

Preparing the visual input. The program finds a requested row-and-column subplot from whitespace between panels, outlines it in red, and adds "outlined in red" to the question. It keeps the full chart, preserving shared labels and legends that a crop could remove. The recorded development comparison gains 20 correct answers out of 2,000; outlining alone was neutral. Tick and layout questions additionally receive larger inputs, with a minimum of 896 visual tokens and a cap of 4,096. Other questions use the existing image preprocessing. The tick-only change gains 15 answers in its development comparison, but the reported uncertainty includes no gain. Enlargement changes the model's representation of existing pixels; it creates no new source detail.

Reading before calculating. Question wording selects an answer procedure. For tick spacing, the model lists numerical labels and Python computes the step; for tick counts, it lists each axis separately and code counts the entries. Counting the listed labels removes a separate long model call. An early development comparison reports 33–35% less time in answer generation and follow-up checks, with other answers unchanged, but uses different hosts. Short checks also test whether requested elements exist, such as whether a proposed title is actually present. The transferred tick wording and formatting add eight correct answers out of 2,000 on the faster Instruct branch without adding a model call. Agents reject the transferred tick parser and wording that asks for the subplot's own axes; the final program retains the eight-agent parser.

Reading layouts without the model. A CPU routine partitions the original image by rows and by columns, accepting only a matching, regular multi-panel grid. It runs alongside the model and replaces the layout answer when accepted; other requests and failed reads keep the model's answer. Its settings were chosen from 108 development variants, and the recorded Instruct comparison gains five answers and loses one out of 2,000. The earlier extension of higher resolution to layout questions was development-neutral and was selected using published validation feedback. That resolution change's validation gain should not be treated as an independently confirmed component effect.

Sharing computation. Beyond the manual baseline's reuse of image features, the program reuses language-model key/value states for each image across questions and follow-up checks. It initially shares this storage across requests, then copies it into separate caches when at most 192 active sequences remain. This avoids the gather overhead that made sharing throughout decoding slower. On the tick-resolution branch, the change reduces peak reserved memory from about 91 to 67 GiB with all 2,000 raw model outputs unchanged. The inherited execution path also removes completed sequences from decoding, replays GPU operations as CUDA graphs, fuses decoder operations, and

overlaps layout and axis-label calls with other work. The final configuration uses batches of 256 and retains the memory guard and small-batch fix.

These intermediate comparisons do not decompose the final gain: they measure answer quality, memory or local runtime under different conditions. Held-out evaluation measures the complete frozen program, including startup time.

D.1.2 Accuracy and speed selections

The speed selection omits the overall program's extra image enlargement, cache-storage optimization, and CPU layout reader.

The accuracy selection first answers every question with Instruct. It marks the requested subplot for both models and applies a minimum image size of 896 visual tokens, capped at 4,096, to every Instruct input. Thinking is also run except for chart titles, x-axis labels, legend names, and subplot layout. Its answer replaces Instruct's when the normalized answers differ and its reasoning ends naturally, without a token-limit cutoff or forced finalization. Otherwise, Instruct's answer is retained. For extreme tick values, an intermediate vote first uses Thinking to resolve disagreement between Instruct's direct answer and tick listing when it matches either, even if its reasoning was cut short. Then two Instruct readings—a reasoning answer and a tick listing—vote with the Thinking answer; the direct answer substitutes when the reasoning pass produces no final answer. When at least two agree, the program uses that answer; otherwise, it retains the preceding hybrid selection. A separate Thinking worker uses the four-agent tick-listing routine to compute total tick counts, replacing the hybrid answer only when a count is obtained. Larger inputs and additional model processes increase runtime and memory use.

D.1.3 Four-agent programs

Answer procedures. Both programs ask the model to read tick labels, then use Python to calculate spacing or count the labels. Instruct also answers legend-count questions by listing and counting legend entries. Combining these procedures raised an early Instruct program from 68.9% to 75.3% on 2,000 development questions. The selected programs retain these procedures alongside checks for absent chart elements and rules for formatting answers. They also enlarge chart inputs; a later Instruct comparison increased development accuracy from 78.5% to 80.8%, at additional execution cost.

Avoiding repeated work. Both programs extend image-feature reuse to shared language-model states across questions about the same chart and stop decoding completed answers. Thinking answers some questions through tick listings before reasoning. In a paired development comparison, this skipped reasoning on 120 of 1,024 questions, reducing batch time by 8–10%.

Reducing execution overhead. Both programs replay decoding with CUDA graphs to reduce Python dispatch; Instruct retains these graphs across calls. Thinking additionally combines query, key, and value projections and fuses normalization operations within each decoder layer. Paired tests in the same process reduced Thinking's call time after warmup by 8–12%. This is a local timing result, not the reduction in total evaluation time. Outputs also change: both versions answer 1,459 of 1,792 development questions correctly, but fusion gains five answers and loses five others.

Unsuccessful changes. Asking Instruct to list ticks in reverse order reduced correct answers from 687 to 634 on 856 development questions; the model often retained the original order. The selected program asks it to stop at the last written tick, gaining only seven of 2,000 development answers without reducing truncated listings. This final prompt edit does not explain the program's overall gain.

D.2 Manual baseline measurements

The naive baseline returns the full response with a 128-token limit, scoring 26.3% for Instruct and 0% for Thinking. Figure 2(a) uses batched execution without compilation, giving 26.6% for Instruct. Extracting Thinking's answer and allowing longer completion repair output handling. Answer-only and applicability prompts remove explanations and check whether the requested element exists. Caching and compilation change Thinking accuracy from 66.4% to 65.9%.

Panel (b) uses the final answer handling and prompts from panel (a). Times average two runs except the initial one-question-at-a-time configuration, measured once; its Thinking time sums four disjoint groups covering 1,000 questions. The compiled Instruct runs take 106.3 and 70.7 s (mean 88.5 s), so their variability is larger than the mean improvement over image reuse alone (93.8 s). These timings exclude loading and warmup. Elsewhere, manual and evolved times share the fresh-process protocol, including loading and compilation.

We reject a bundled resize and resolution increase: Instruct rises from 55.2% to 55.4%, but Thinking falls from 66.4% to 64.4%. CPU image metadata preserves outputs without a consistent timing benefit.

D.3 Original supplied questions

We run the submitted programs on the assignment's original 128 descriptive questions from 32 charts (Table 8). This reports accuracy on the supplied examples and checks the delivered code against recorded development results. These charts belong to our development set and were available during optimization, so the scores do not measure generalization.

All seven programs complete the fresh evaluation without inference errors or recorded worker fallbacks, using their frozen code and settings. Five scores match the development-run subset; manual Thinking answers one fewer question correctly and evolved Thinking answers two fewer. Batch composition differs between the evaluations; we have not isolated the cause of these small differences. No tuning or selection changes follow this check.

The check uses the delivered batch evaluator on GH200; the supplied reference evaluator's single-question interface remains unmeasured. The two sets of measurements are recorded in `evidence/supplied_questions.json` and `evidence/supplied_questions_fresh.json`.

Table 8: **Accuracy on the supplied questions.** Both columns score the same 128 questions with the same frozen programs. The first extracts this subset from larger development evaluations; the second reruns only these questions through the delivered batch evaluator. Blocks follow Table 1.

Program	Subset from development runs (%)	Fresh 128-question run (%)
Manual Instruct	65.6	65.6
Manual Thinking	73.4	72.7
Evolved Instruct	87.5	87.5
Evolved Thinking	85.9	84.4
Best accuracy (both)	**91.4**	**91.4**
Best speed (Instruct)	90.6	90.6
Best overall (Instruct)	89.1	89.1